

Colony-Bounded Hybrid Learning for Cold-Start Fog Service Placement

Rouhollah Nabati, Abdulbaghi Ghaderzadeh

Department of Computer Engineering, *Islamic Azad University, Sanandaj Branch*, Sanandaj, Iran
rnabati@gmail.com

Department of Computer Engineering, *Islamic Azad University, Sanandaj Branch*, Sanandaj, Iran
b.ghaderzadeh80@gmail.com

Abstract

When fog nodes arrive under burst load and membership change, a placement policy that assumes a ready hierarchy or a near-global host list either spends heavily on control traffic or fails early. We study the complementary problem: learning to rank hosts *inside* an already bounded cold-start control plane. **Dyna-Bound** keeps DynaCol colony formation, FCM handover, and CRT/GRT tables unchanged, and replaces only local L2 ranking. An online scorer—a small GraphSAGE encoder or a flat vector—is mixed with the deterministic DCBO objective and is switched off once the fabric is large enough for exact DCBO. On iFogSim2 multipipe workloads ($N=100$, 20 paired trials) both hybrids cut mean SLA violation relative to DCBO at matched CRT/GRT overhead; paired gaps remain significant after Holm correction (Cohen’s d roughly 0.6–1.1). The two encoders are statistically tied, so the result is about bounded hybrid ranking, not a general GNN advantage. Ablations show that widening the action set beyond CRT improves absolute SLA at the price of the visibility the colony tables withhold; at $N=1000$ the methods converge under the large-scale gate.

Keywords: fog computing, service placement, hybrid learning, cold start, self-organizing systems, graph neural networks, CRT/GRT, iFogSim2

1 Introduction

Fog and edge placement must keep end-to-end delay inside SLA limits while demand, membership, and link quality change [7, 13, 33, 50]. Under burst arrivals and node departures, controllers that wait for a global snapshot, or that assume a finished hierarchy, either flood the network with control messages or miss early requests [2, 18, 45]. Learning-based placers adapt under shifting load [25, 39, 44], but many still treat almost every host as a legal action and rarely account for the messaging cost of that choice [1, 23].

Our prior control plane, DynaCol, already forms colonies from an empty state, elects and hands over an FCM, and keeps CRT/GRT neighbor tables bounded. DCBO is its default L2 scorer, with sticky L1 reuse and cloud L3 fallback. What it does not provide is an online learner that is *required* to choose inside those same tables. That constraint is the focus of this paper: if the control plane must stay cheap and decentralized, can hybrid learning still improve SLA without opening a near-global host set?

Dyna-Bound leaves formation, handover, CRT/GRT, L1, and L3 as in DynaCol and changes only L2 ranking (Fig. 1). A small online scorer is added to the inverted DCBO utility; the learning weight falls with fog count and is set to zero beyond a live-node gate so large fabrics return to exact DCBO. We try two scorers with the same hybrid rule: a HID=16 GraphSAGE encoder on RTT affinity and service-DAG context (with optional offline import and fine-tune), and a flat vector without message passing (Fig. 2). GraphSAGE is the structured default; it is not offered as proof that GNNs always beat linear scores on CRT neigh-

borhoods.

After cold start the action set satisfies $|A_t| \leq K_{\text{crt}}$, so the overlay is part of the learning problem rather than an external assumption. We evaluate on iFogSim2 under normal, burst, and churn loads against DCBO at matched CRT/GRT overhead, with a tabular ϵ -greedy learner and lightweight CRT-bounded DQN/PPO variants as additional colony-plane references, plus named flat baselines for absolute SLA context. The primary statistical evidence is a 20-trial multipipe grid with paired tests, Holm adjustment, bootstrap intervals, and Cohen’s d , supported by sensitivity checks, an $N=1000$ run, CRT/offline/ β ablations, and a separate node-failure stress test.

Contributions.

1. A colony-bounded MDP in which feasible L2 actions are the CRT top- K (and limited GRT escalation), so cold-start visibility is a hard learning constraint rather than a soft preference.
2. A hybrid L2 rule that mixes an online score with DCBO and hands ranking back to exact DCBO at large scale; the main empirical claim is Hybrid $\succ$ DCBO at matched control overhead on the multipipe journal grid.
3. Two interchangeable scorers under that rule (GraphSAGE and vector) plus colony-plane reference learners (tabular ϵ -greedy, lightweight DQN/PPO), used to separate the hybrid design from any single encoder story.

4. An empirical package on iFogSim2: $n=20$ multipipe tests with effect sizes, scale and sensitivity checks, CRT visibility ablations, flat-baseline Pareto context, and hard router-failure trials at 5–20%.

2 Related Work

2.1 Fog and edge service placement

Fog computing extends the cloud toward latency-sensitive, geo-distributed IoT workloads [7, 13, 33, 35]. Surveys of service placement and offloading catalogue latency, energy, and cost objectives under fixed or centralized controllers [2, 3, 18, 42, 45, 50]. Classical and heuristic mappers assign modules along fog–cloud paths (Edgeward-style resource-aware placement, FogTorch / FogTorchII QoS matching, optimized IoT placement, and FogPlan Min-Cost) [9, 10, 40, 41, 46, 51]. Metaheuristics and ILP-style workload allocation further improve mapping quality once a topology is given [4, 5, 16, 19]. These lines assume an already organized fabric or a global/central view; they do not form a self-organizing overlay from an empty colony state, nor do they treat a bounded neighbor table as a hard action set for online learning.

2.2 Learning-based offloading and placement

DQN/PPO-style and broader ML placers adapt under changing demand [1, 25, 32, 39, 44]. Recent DRL work targets healthcare/fog stacks, energy-aware modules, multi-tier containers, microservice instance selection, and IoT job deployment [8, 11, 17, 24, 47]. Resource-management and orchestration surveys still note large action spaces and the assumption of an organized fabric [15, 23, 31, 37]. Dyna-Bound keeps $|A_t|$ inside CRT/GRT top- K , mixes learning with DCBO, and reports the same table overhead as the colony control plane. The point is not another open-set DRL placer, but a learner that cannot leave the cold-start view.

2.3 Self-organizing and hierarchical fog control

Overlay formation, manager election, and hierarchical module managers separate structure from placement [6, 29, 36]. Evolutionary and mobility-aware schedulers improve QoS once a hierarchy exists [19], but learning is seldom constrained to the colony view as a hard limit. Here formation and FCM handover stay with DynaCol; learning only re-ranks inside that view.

Table 1: Where Dyna-Bound sits relative to common placement families.

Method family	Cold-start	Bounded view	Hybrid w/ determ. obj.	Local graph/DAG
Classical / GA / ILP	Rarely	No	Sometimes	Rarely
FogPlan / Edgeward / Static-DCBO	No/partial	No/fixed	No	No
DRL placers (DQN/PPO)	Rarely	Rarely	Rarely	Sometimes
GNN-RL placers	Rarely	Rarely	Rarely	Yes
DynaCol / DCBO	Yes	Yes	N/A	No
Dyna-Bound	Yes	Yes	Yes	Yes

2.4 Graph learning on infrastructure and service graphs

Message-passing encoders (GCN, GraphSAGE, GIN, GAT) provide inductive structure models for neighborhoods and dependency graphs [21, 26, 48, 49]. GNN-RL placers encode service and/or MEC graphs for multi-objective mapping and adaptive microservice placement, including dual-GNN DDPG designs [12, 27]. Those peers typically optimize over broader device sets than a cold-start CRT. Our CRT neighborhoods are small and RTT-structured, so local message passing is cheap if the action set stays colony-bounded. On the multipipe journal grid the GraphSAGE and vector scorers land statistically together; the hybrid rule, not the encoder family, is what we defend against DCBO.

2.5 Relation to DynaCol

DynaCol/DCBO is the prior control plane and the matched-overhead baseline [34]. Cold-start formation and FCM handover are not restated as new results. The companion DynaCol write-up is cited as an unpublished control-plane artifact; the reused algorithms are reprinted in Appendix A so this paper stays readable on its own. What is new here is the colony-bounded L2 MDP, the hybrid GraphSAGE/vector ranking with a scale-aware blend, the colony-plane reference learners, and the multipipe evaluation package.

We use iFogSim / iFogSim2 for simulation [20, 30] (EdgeCloudSim is a related alternative [43]) and Holm’s procedure for multiple comparisons [22], with Cohen’s d for paired effect sizes [14].

3 Background: DynaCol Control Plane

Only the reused pieces are summarized here [34]; pseudocode appears in Appendix A.

Nodes join by query/advertise until colonies appear. An FCM is elected by potency and updated with ε -greedy probes and a cooldown on handover. Each FCM keeps a CRT of member capacity, availability, RTT, and attractiveness (default top- $K_{\text{crt}}=5$); GRT summaries allow limited cross-colony search (default top- $K_{\text{grt}}=3$) without a full host list. Placement is three-tier: sticky L1 reuse if the current host still meets SLA thresholds; DCBO L2 scoring over

CRT then GRT; cloud L3 if both fail. Dyna-Bound replaces only the L2 ranking inside CRT/GRT top- K (Fig. 1).

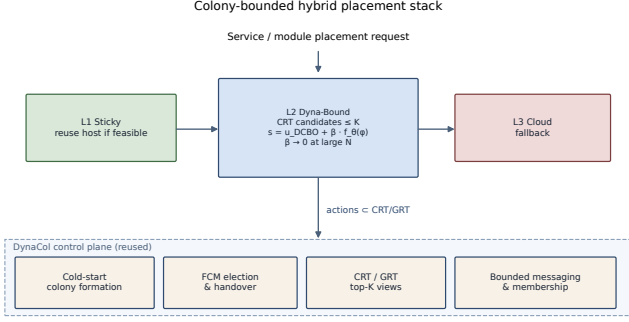


Figure 1: Placement stack on the reused DynaCol control plane: L1 sticky reuse, L2 Dyna-Bound over CRT/GRT, L3 cloud fallback.

4 Colony-Bounded Hybrid Learning

4.1 MDP

On each FCM reconcile, state S_t is the local CRT (size $\leq K_{\text{crt}}$), optional GRT summaries, and service/DAG features. Action A_t picks a feasible CRT host or escalates to GRT/L3. The hard limit $|A_t| \leq K_{\text{crt}}$ is intentional: the control plane is part of the learning method.

Host features (dimension 8) and service/context features (dimension 8) are listed in Table 2. Continuous resource channels are scaled by the colony-local maximum available value; RTT is clipped by 100 ms; deadline context is clipped by 500 ms. No learned feature weights are applied before the encoder; the vector ablation zeros DAG topology and co-location channels so message passing remains the main encoder difference.

Table 2: Candidate features used online (no PCA or other dimensionality reduction).

Group	Feature	Normalization
Host	CPU, RAM, storage, BW	available / colony max
Host	RTT to FCM	$\min(1, \text{RTT}/100)$
Host	Attractiveness	clipped to $[-1, 1]$
Host	Neighbor pressure, edge entropy	GraphSAGE only; else 0
Context	Demand CPU/RAM	demand / colony max
Context	Deadline	$\min(1, D/500)$
Context	In/out degree, depth, source/sink	DAG norms (GraphSAGE); Vector zeros
Context	Producer-RTT pull, fan-in, co-location	GraphSAGE; Vector zeros co-location

Reward. Let r_{attr} be the attractiveness reward used by DCBO and r_{dcbo} the same signal at the DCBO-preferred CRT host. The shaped online target is

$$R_t = \text{clip}_{[0,1]}(0.5 + 0.5(r_{\text{attr}} - r_{\text{dcbo}})). \quad (1)$$

For GraphSAGE, a graph-SLA score (producer-RTT pull, co-location under fan-in) further biases the contrastive update. Vector uses the same R_t with a weak linear TD-style update on the chosen feature vector.

4.2 Hybrid scoring

$$s(i) = u_{\text{DCBO}}(i) + \beta \cdot f_{\theta}(\phi(i)), \quad (2)$$

with u_{DCBO} the inverted DCBO objective. We take $\arg \max_i s(i)$ and allow a small ε -greedy rate while n_f is below the large-scale gate. The blend β falls with fog count (default 0.60 on multipipe at small N) and is set to zero for $n_f \geq 450$, so L2 becomes exact DCBO.

4.3 Encoders

The vector policy (`colony_rl`) concatenates host channels with service/DAG scalars and scores with a linear head (learning rate 0.05). The GraphSAGE policy (`colony_gnn`) builds a structure-aware affinity graph on CRT candidates: RTT base $A_{ij} \propto 1/(1+\text{RTT}_{ij}/20)$ plus coloc and producer-host pull (RTT-only ablation via a system flag), then a two-layer GraphSAGE-style update (hidden width 16)

$$h' = \tanh(W_{\text{self}}h + W_{\text{neigh}}Ah), \quad (3)$$

then reads out from $\text{concat}(h, \text{context})$ (learning rate 0.04). Offline JSON weights can be loaded and fine-tuned online (Fig. 2).

A tabular ε -greedy baseline (`dr1`) keeps cold-start CRT/GRT formation but replaces L2 with attractiveness selection ($\varepsilon=0.15$), without a neural encoder or DCBO hybridization. We also include two lightweight CRT-bounded neural references (`colony_dqn`, `colony_ppo`): shared one-hidden-layer scorers on the same flat features as the vector path, with a one-step TD update (DQN-style) or a clipped likelihood-ratio nudge with a moving baseline (PPO-style). They are not full open-set deep RL stacks; they only ask whether a small neural ranker inside CRT can match the hybrid under the same action bound.

4.4 Online updates and offline import

After t^* is chosen, GraphSAGE applies a centered advantage update $R_t - \bar{R}$ with a soft contrastive push toward the graph-SLA-best CRT index when available; Vector applies a delta rule on the chosen features toward R_t . Attractiveness decay/update mirrors DCBO.

Dyna-Bound encoders: Vector ablation vs GraphSAGE

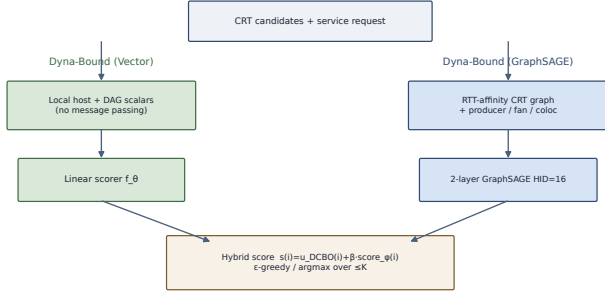


Figure 2: Dyna-Bound Vector and GraphSAGE encoder pipelines into the same hybrid score (2).

Offline path. Exploratory `colony_gnn` runs log CRT tensors (host matrix, context, adjacency, action, reward, optional DCBO base scores). Outside the simulator we train a GIN-style encoder [49] matching the Java layout with **reward-advantage behavioral cloning plus structure ranking** (not classic PPO), in the spirit of imitation / offline policy learning from logged trajectories [28, 38]: weighted NLL on the logged action, softplus ranking between structure-best/worst hosts, an optional ranking term against the inverted DCBO base vector, and a small entropy bonus. Exported JSON tensors load into the online GraphSAGE; champion and journal grids enable online fine-tuning.

4.5 Complexity

Let $K=K_{\text{crt}}$ (default 5), host/context width $d=8$, and GraphSAGE hidden width $h=16$. Per reconcile after cold start:

- **Query.** Read at most K CRT rows and optional K_{grt} GRT summaries: $O(K + K_{\text{grt}})$.
- **Scoring.** Feature build is $O(Kd)$. Vector scoring is $O(Kd)$. GraphSAGE with dense CRT affinity is $O(K^2d + Kh(d+h))$ and stays constant-order for fixed K, h . Hybrid blend with DCBO utility is $O(K)$.
- **Update.** Vector delta rule $O(d)$; GraphSAGE centered advantage on the chosen neighborhood $O(Kh(d+h))$.

Total decision time is therefore $O(K)$ in the CRT size with fixed d, h , not $O(N)$ in the fog population. Memory is $O(K)$ for CRT, $O(K_{\text{grt}})$ for GRT, plus $O(dh + h^2)$ encoder weights (HID=16). Offline GIN training is done outside the control loop; online fine-tune uses the same small parameter set. Control messaging remains the CRT/GRT traffic of DynaCol.

Algorithm 1 Colony-bounded hybrid reconcile (sketch)

```

1: procedure RECONCILE( $r, \text{fcm}, \text{current}$ )
2:   if  $\beta \approx 0$  then
3:     return DCBO.reconcile(...)
4:   end if
5:   if L1 sticky OK then
6:     return current
7:   end if
8:   if CRT enabled then
9:      $C \leftarrow$  top- $K$  feasible CRT entries
10:    pick  $i^*$  by hybrid score on  $\phi(C)$ ; update  $\theta$ 
11:    return host( $i^*$ )
12:  end if
13:  if GRT enabled then
14:    escalate via ranked GRT summaries
15:  end if
16:  return cloud
17: end procedure

```

5 Experimental Setup

Simulations run in iFogSim2 on the shared DynaCol runtime [20, 30]. We compare `dcbo`, `colony_r1` (Vector), `colony_gnn` (GraphSAGE), tabular `dr1`, and the lightweight CRT-bounded `colony_dqn/colony_ppo` scorers above, plus flat `static_dcbo`, `fogplan` [51], and `greedy`. Edgeward-style module mapping [46] was dropped on multipipe after single trials exceeded 25 minutes. The DQN/PPO variants never expand the action set beyond CRT top- K ; they are colony-plane references, not re-implementations of open-host DQN/PPO placers from the broader literature.

5.1 Topology and workload

The fabric is a four-tier iFogSim hierarchy: one cloud, a proxy, fog routers, and camera leaves. Fog routers use about 2800–5600 MIPS and 4 GB RAM; cameras about 500 MIPS and 1 GB RAM; uplink/downlink bandwidth on fog links is 10 Gbps-class in the simulator defaults. Target sizes are $N \in \{100, 300, 500, 1000\}$ under normal, burst, and churn. We also inject hard fog-router failures at mid-run for fractions 5%, 10%, and 20% of router nodes (`node_fail05/10/20`), reporting SLA% after failure plus hosts failed, modules displaced, and approximate recovery time to the first post-failure reconcile. The surveillance DAG is the shallow camera pipeline; the multipipe DAG adds fan-out, skip, and join edges (Fig. 3).

5.2 SLA violation metric

Let L_j be the end-to-end loop delay (ms) of completed request j , and let M be the number of completed loop

samples in a trial. The deadline (ms) is

$$D(N, s, \sigma) = \sigma \cdot \alpha(s)(60 + N/10), \quad (4)$$

where $\alpha(\text{normal})=1$, $\alpha(\text{burst})=0.85$, $\alpha(\text{churn})=0.88$, and σ is the deadline scale ($\sigma=10$ on the locked multipipe grids; $\sigma=1$ on surveillance unless noted). Request j violates the SLA if $L_j > D$; the reported violation rate is

$$\text{SLA\%} = 100 \cdot \frac{1}{M} \sum_{j=1}^M \mathbf{1}[L_j > D]. \quad (5)$$

Sensor periods are 5 ms (normal/churn) and 1 ms (burst). We also report P95 loop delay, normalized control overhead, and colony count.

5.3 Statistics

The tuning lock uses 5 trials; the journal $N=100$ multipipe grid uses **20** paired trials. For each method pair we report mean $\pm$ std of SLA%, a paired t -test, Holm-adjusted p -values across the comparison family [22], percentile bootstrap 95% CIs on the paired mean difference, and Cohen’s d on paired differences [14]. $N=1000$ uses 10 trials; flat baselines use 5 trials. One-factor sensitivity sweeps (β , deadline scale, ε) use 5 Burst trials. Locked settings: $\text{HID}=16$, $\beta=0.60$, $\varepsilon=0.02$, offline weights with fine-tune. Companion 5-trial ablations isolate the CRT action bound, offline import, and $\beta=0$ DCBO delegation (Sec. 6.8).

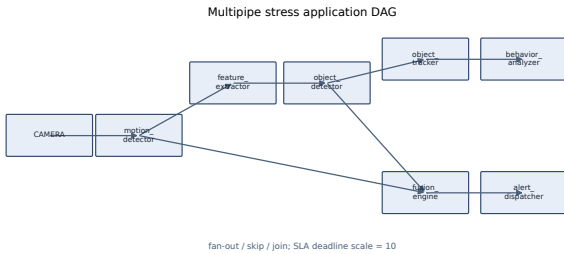


Figure 3: Multipipe application DAG used in the champion and journal grids.

6 Results

6.1 Journal multipipe grid ($n=20$)

Table 3 and Fig. 4 are the primary multipipe result and the only grid we use for the Hybrid-versus-DCBO statistical claim. Both hybrids beat DCBO on every load at matched overhead (≈ 1770 normalized messages; ≈ 7.65 colonies). Table 4 lists paired differences versus DCBO with bootstrap 95% CIs, Cohen’s d , and Holm-adjusted p -values; all hybrid–DCBO intervals exclude zero and clear $\alpha=0.05$ after Holm correction (Fig. 5). GraphSAGE and

Vector means sit within a tenth of a point of each other and are not distinguishable (CIs cover zero). The absolute gaps are modest (about 0.6–1.1 percentage points): they matter because they arrive without extra CRT/GRT traffic, not because they rewrite the SLA scale. Tabular ε -greedy DRL records still lower SLA means at slightly lower overhead (≈ 1501) with the same colony count. We keep it in the table as a colony-plane reference; Section 7 explains why the hybrid is still the submitted design.

Table 3: Multipipe journal grid: mean $\pm$ std SLA % (20 trials, $N=100$). GS = GraphSAGE encoder; V = vector ablation (both under Dyna-Bound).

N	Scenario	Dyna-Bound (GS)	Dyna-Bound (V)	DCBO	
100	Normal	10.87 $\pm$ 2.29	10.74$\pm$2.39	11.49 $\pm$ 2.48	7
100	Burst	25.60 $\pm$ 2.49	25.55$\pm$2.43	26.62 $\pm$ 2.41	2
100	Churn	22.26 $\pm$ 2.32	22.25$\pm$2.64	23.39 $\pm$ 2.47	1

Table 4: Paired SLA differences vs DCBO on the multipipe journal grid ($N=100$, 20 trials): mean Δ , bootstrap 95% CI, Cohen’s d , Holm p .

Scenario	Contrast	Mean Δ	95% CI	d	Holm p
Normal	GS–DCBO	−0.62	[−1.04, −0.21]	−0.61	0.025
Normal	V–DCBO	−0.74	[−1.23, −0.32]	−0.69	0.012
Burst	GS–DCBO	−1.02	[−1.41, −0.65]	−1.14	< 0.001
Burst	V–DCBO	−1.06	[−1.76, −0.34]	−0.64	0.022
Churn	GS–DCBO	−1.12	[−1.65, −0.61]	−0.86	< 0.001
Churn	V–DCBO	−1.13	[−1.75, −0.53]	−0.81	0.002

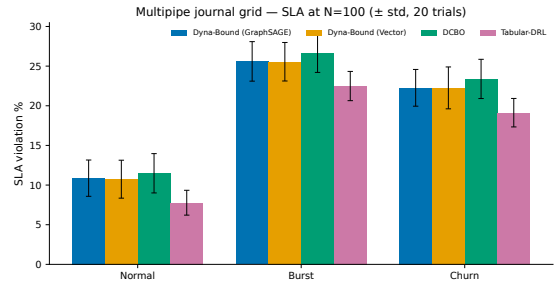


Figure 4: Multipipe journal-grid SLA at $N=100$ (20 trials).

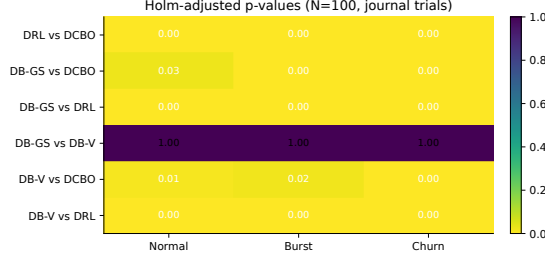


Figure 5: Holm-adjusted p -values for paired SLA tests at $N=100$ (20 trials).

6.2 Tuning lock and offline path

The 5-trial multipipe lock (Table 5, Fig. 6) fixed offline weights and hyperparameters before the journal grid; we treat it as a tuning lock, not the primary statistical claim. Without imported weights, an earlier online-only medium put Vector slightly ahead of GraphSAGE while both beat DCBO (Table 6), so the offline structure-weighted path is part of the named GraphSAGE pipeline rather than a free online GNN-over-Vector gap.

Table 5: Multipipe tuning lock: mean SLA % (5 trials, $N=100$).

N	Scenario	Dyna-Bound (GS)	Dyna-Bound (V)	DCBO
100	Normal	10.64	10.98	11.48
100	Burst	24.89	25.35	26.13
100	Churn	22.31	22.43	23.21

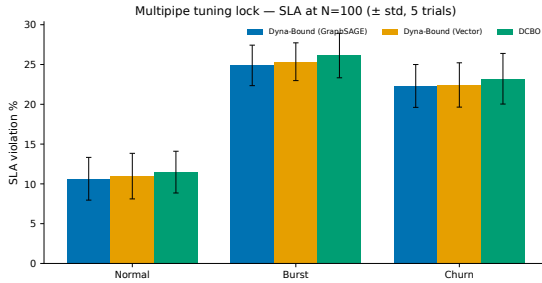


Figure 6: Multipipe tuning-lock SLA at $N=100$ (5 trials).

Table 6: Earlier online-only multipipe medium (context; pre-lock).

N	Scenario	Online DB (GS)	Dyna-Bound (V)	DCBO
100	Normal	10.34	10.25	11.21
100	Burst	24.39	24.09	26.42
100	Churn	21.15	21.13	23.04

6.3 Surveillance DAG

Table 7 and Fig. 7 summarize the surveillance grid. At $N=100$ the hybrid (Vector and GraphSAGE coincide on this shallow app) lowers mean SLA against DCBO at matched overhead. At $N \in \{300, 500\}$ the curves meet because the blend gate hands ranking back to DCBO (Fig. 8).

Table 7: Surveillance DAG: mean SLA violation % (5 trials).

N	Scenario	Hybrid	DCBO
100	Normal	11.51	13.31
100	Burst	14.02	14.78
100	Churn	14.58	15.37
300	all	$\approx 0.48-0.51$	$\approx 0.48-0.50$
500	all	$\approx 0.57-0.59$	$\approx 0.57-0.59$

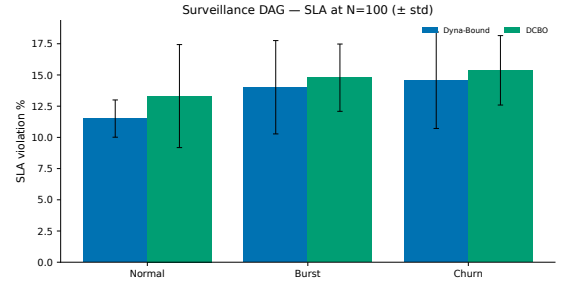


Figure 7: Surveillance SLA at $N=100$ (mean $\pm$ std, 5 trials).

6.4 Scale, overhead, and sensitivity

Fig. 8 shows surveillance mean SLA versus target size. Fig. 9 shows matched overhead for colony-plane hybrids and DCBO. One-factor Burst sweeps (Fig. 10) place $\beta=0.60$ and $\varepsilon=0.02$ in the interior of mild SLA variation; deadline scale shifts absolute SLA sharply (as expected) while hybrid-over-DCBO direction holds at scales 8 and 12. The gate uses the live fog count ($n_f \geq 450 \Rightarrow$ exact DCBO).

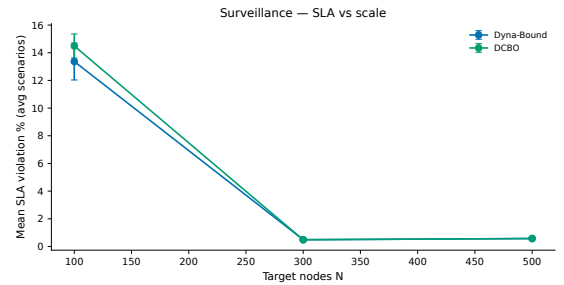


Figure 8: Surveillance mean SLA versus target size N (averaged over scenarios).

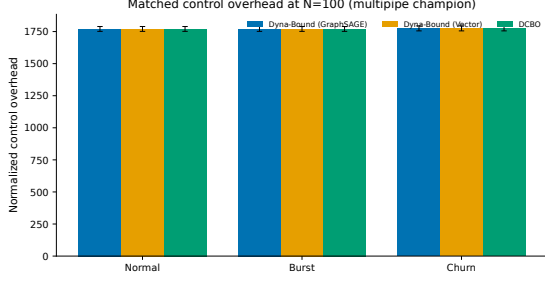


Figure 9: Normalized control overhead for colony-plane methods ($N=100$). GraphSAGE, Vector, and DCBO remain matched.

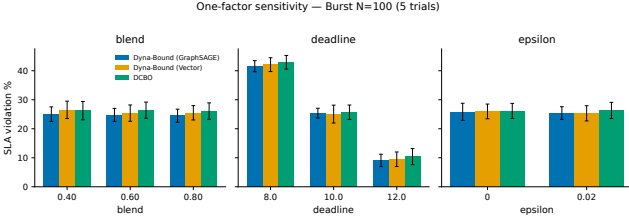


Figure 10: One-factor sensitivity on Burst $N=100$ (5 trials): blend, deadline scale, and ϵ .

6.5 Large- N check

At $N=1000$ (Table 8, Fig. 11) absolute SLA is low and overhead matches (≈ 2112 – 2140). Burst/churn means still favor GraphSAGE; Holm tests after correction remain mostly non-significant, consistent with near-parity under the large- N blend gate.

Table 8: Multipipe at $N=1000$: mean SLA % (10 trials).

Scenario	Dyna-Bound (GS)	Dyna-Bound (V)	DCBO	Overhead
Normal	0.82	0.80	0.81	2112
Burst	2.53	3.13	3.59	2112
Churn	1.94	2.27	2.70	2140

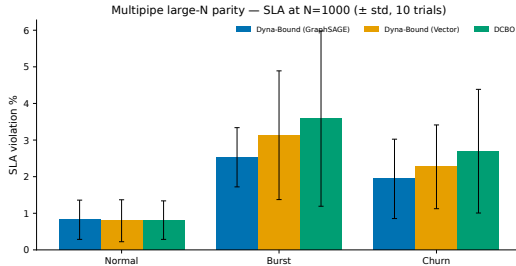


Figure 11: Multipipe SLA at $N=1000$ (mean $\pm$ std, 10 trials).

6.6 CRT-bounded DQN/PPO references and hard node failures

Table 9 is a separate five-trial multipipe lock that adds the lightweight CRT-bounded DQN/PPO scorers. Means for GraphSAGE, Vector, and DCBO need not match Table 3: different trial counts and seeds, and this table is only for colony-plane neural references, not for the Holm claim. On this lock the hybrids again sit near or below DCBO; DQN and PPO are more variable (larger std) and do not dominate the hybrid under the same CRT bound.

Hard fog-router failures at mid-run (5%, 10%, 20% of router nodes) trigger an L3 reconcile. Table 10 reports SLA under that stress together with how many hosts were taken down, how many mapped modules sat on them, and the simulated time until the first post-failure reconcile. Host counts and reconcile latency are largely properties of the shared control plane, so they look similar across colony-plane methods; the useful comparison is the SLA column in Table 9. Churn remains the membership scenario; failure is a separate hard-down test.

Table 9: CRT-bounded DQN/PPO vs hybrids and DCBO (multipipe, $N=100$, 5 trials; mean $\pm$ std SLA %). Separate seed lock from Table 3.

Scenario	GS	V	DCBO	DQN	PPO
Normal	9.96 $\pm$ 1.48	10.17 $\pm$ 2.09	11.33 $\pm$ 2.69	12.55 $\pm$ 3.24	13.03 $\pm$ 3.85
Burst	23.96 $\pm$ 1.00	23.86 $\pm$ 1.41	26.24 $\pm$ 2.50	23.36 $\pm$ 5.43	26.88 $\pm$ 6.12
Churn	21.47 $\pm$ 1.61	20.75 $\pm$ 1.11	23.13 $\pm$ 2.37	21.74 $\pm$ 5.35	24.15 $\pm$ 5.87
Fail 5%	21.07 $\pm$ 1.48	20.10 $\pm$ 1.10	23.13 $\pm$ 2.43	22.00 $\pm$ 5.06	23.87 $\pm$ 5.71
Fail 10%	21.00 $\pm$ 1.26	20.80 $\pm$ 1.42	23.45 $\pm$ 3.28	22.41 $\pm$ 5.24	24.06 $\pm$ 5.93
Fail 20%	20.96 $\pm$ 1.73	21.19 $\pm$ 1.34	22.94 $\pm$ 3.17	22.33 $\pm$ 4.96	23.66 $\pm$ 5.64

6.7 Named baselines and Pareto view

FigPlan, Greedy-Nearest, and Static-DCBO can post lower multipipe SLA than the colony hybrids (Table 11, Fig. 12), but their overhead and colony counts are not comparable. Fig. 13 plots SLA against normalized overhead on Burst: flat methods sit cheaper; colony-plane methods cluster together; tabular DRL sits between them on cost while leading SLA inside the cold-start view. We therefore defend a within-plane gain over DCBO at matched hybrid/DCBO overhead, not a win over every flat placer.

Table 10: Hard fog-router failures (multipipe, $N=100$, 5 trials): mean $\pm$ std hosts taken down, modules that had been mapped onto them, and simulated ms to the first post-failure reconcile (shared control-plane timing; compare SLA in Table 9).

Fail %	Method	Hosts	Modules lost	Recovery (ms)
5	Dyna-Bound (GS)	1.0 $\pm$ 0.0	4.8 $\pm$ 3.0	5.0 $\pm$ 0.0
5	Dyna-Bound (V)	1.0 $\pm$ 0.0	4.8 $\pm$ 3.0	5.0 $\pm$ 0.0
5	DCBO	1.0 $\pm$ 0.0	4.8 $\pm$ 3.0	5.0 $\pm$ 0.0
5	Colony-DQN	1.0 $\pm$ 0.0	4.8 $\pm$ 3.0	5.0 $\pm$ 0.0
5	Colony-PPO	1.0 $\pm$ 0.0	4.8 $\pm$ 3.0	5.0 $\pm$ 0.0
10	Dyna-Bound (GS)	2.0 $\pm$ 0.0	8.4 $\pm$ 4.1	5.0 $\pm$ 0.0
10	Dyna-Bound (V)	2.0 $\pm$ 0.0	8.4 $\pm$ 4.1	5.0 $\pm$ 0.0
10	DCBO	2.0 $\pm$ 0.0	8.4 $\pm$ 4.1	5.0 $\pm$ 0.0
10	Colony-DQN	2.0 $\pm$ 0.0	8.4 $\pm$ 4.1	5.0 $\pm$ 0.0
10	Colony-PPO	2.0 $\pm$ 0.0	8.4 $\pm$ 4.1	5.0 $\pm$ 0.0
20	Dyna-Bound (GS)	4.0 $\pm$ 0.0	16.8 $\pm$ 1.8	5.0 $\pm$ 0.0
20	Dyna-Bound (V)	4.0 $\pm$ 0.0	16.8 $\pm$ 1.8	5.0 $\pm$ 0.0
20	DCBO	4.0 $\pm$ 0.0	16.8 $\pm$ 1.8	5.0 $\pm$ 0.0
20	Colony-DQN	4.0 $\pm$ 0.0	16.8 $\pm$ 1.8	5.0 $\pm$ 0.0
20	Colony-PPO	4.0 $\pm$ 0.0	16.8 $\pm$ 1.8	5.0 $\pm$ 0.0

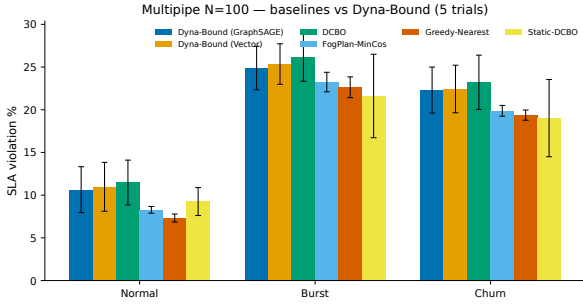


Figure 12: Flat baselines versus colony-bounded methods on multipipe $N=100$ (5 trials).

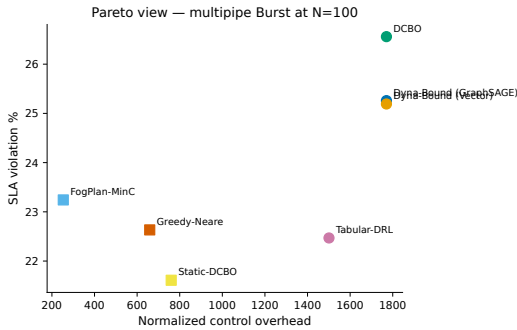


Figure 13: Pareto view: SLA vs normalized control overhead (Burst, $N=100$).

Table 11: Multipipe $N=100$: flat baselines (5 trials) beside the tuning-lock colony means. Overhead is not matched across planes.

Scenario	FogPlan	Greedy	Static	DB (GS)	DB (V)	DCBO
Normal	8.28	7.32	9.25	10.64	10.98	11.48
Burst	23.24	22.63	21.61	24.89	25.35	26.13
Churn	19.88	19.37	19.02	22.31	22.43	23.21
Overhead	253	659	760	≈ 1769	≈ 1769	≈ 1769
Colonies	1	1	19	≈ 7.6	≈ 7.6	≈ 7.6

6.8 Companion ablations

Table 12 reports three 5-trial multipipe checks against the bounded GraphSAGE smoke lock (same seeds/props except the stated factor).

CRT action-set ablation. Raising the L2 pool from CRT top- $K=5$ to a global fog top- $K=50$ (**crtAblate**) cuts mean SLA violation by roughly 4–5 points while the reported CRT/GRT overhead stays matched. The bound is therefore a deliberate visibility limit under decentralized tables. Giving the simulator a global candidate list without charging discovery traffic shows how much SLA the CRT withholds; the method keeps that limit to preserve bounded messaging.

Offline import. Removing imported GraphSAGE weights (online-only fine-tune from random init) raises GraphSAGE SLA by about 0.7–1.8 points versus the locked import+FT path, so the offline BC+ranking stage contributes measurable quality under the same online hybrid.

$\beta=0$ delegation. Forcing $\beta=0$ routes L2 to exact DCBO; GraphSAGE and DCBO means stay within a fraction of a point, confirming the scale-aware hand-back path.

Table 12: Companion ablations (multipipe, $N=100$, 5 trials): mean SLA %. Bounded GraphSAGE is the $n=5$ smoke lock; CRT-ablate expands L2 to global top-50.

Scenario	Bounded DB (GS)	CRT-ablate DB (GS)	No-offline DB (GS)
Normal	10.32	5.46	11.38
Burst	24.88	20.93	26.68
Churn	22.39	17.24	23.05

7 Discussion

The question this paper answers is narrow: once CRT/GRT traffic is fixed by DynaCol, does hybrid L2 ranking beat plain DCBO? On the $n=20$ multipipe grid the answer is yes (Table 3). Paired gaps survive Holm correction, with Cohen’s d for GraphSAGE–DCBO about 0.61–1.14 [14]. The effect is medium in the statistical sense

and small on the absolute SLA axis (roughly one percentage point). That is still useful in this setting because the gain does not buy extra control messages: the hybrids and DCBO share the same overhead band near 1770 normalized units. Formation, FCM handover, and table size limits remain DynaCol’s; only the L2 score changes.

GraphSAGE and the vector scorer finish essentially tied on that grid. We keep GraphSAGE as the structured default because it carries CRT affinity edges and an offline import path [21, 38, 49], but the hybrid-versus-DCBO gap does not depend on message passing for these shallow CRT neighborhoods. Calling the method “colony-bounded hybrid learning” is therefore more accurate than selling it as a GNN win.

Tabular ϵ -greedy DRL often records the lowest SLA means in the colony view, sometimes with slightly less overhead. We do not hide that. The submitted design is still the hybrid for three practical reasons. First, it stays anchored to DCBO, so when the live fog count crosses the large- N gate ranking falls back to the same deterministic rule operators already trust. Second, the offline BC+ranking import gives GraphSAGE a warm start that the tabular learner does not use. Third, the hybrid is built to keep CRT/GRT overhead matched to DCBO by construction; the tabular policy is a useful reference, not a drop-in replacement for that contract. Against open-set GNN-RL placers that score much wider device sets [12, 27] we make no ranking claim.

The CRT ablation (Table 12) is easy to misread. Opening L2 to a global fog top-50 without charging discovery traffic lowers SLA violation by several points. That shows how much QoS the colony bound withholds; it does not show that a larger table is “free.” A fairer follow-up would bill inventory messages. Setting $\beta=0$ recovers DCBO within a fraction of a point, which checks the hand-back path. Removing offline weights hurts GraphSAGE by about 0.7–1.8 points under the same online hybrid.

Several grids appear in the paper on purpose. The Holm claim uses only Table 3. Table 9 is a later five-trial lock for lightweight CRT-bounded DQN/PPO references; means for the shared methods can drift with seeds and should not be mixed into the $n=20$ tests. On that lock DQN/PPO do not overturn the hybrid, and their trial-to-trial spread is larger. They are small shared-MLP scorers with one-step updates, not full open-host deep RL systems.

Flat baselines in Table 11 and Fig. 13 answer a different question: what SLA is possible if one ignores cold-start colony cost. FogPlan and related methods can look better on raw SLA while spending far less (or structuring the fabric differently). That comparison is informative and intentionally unmatched.

Limits. Evidence is simulation-only (iFogSim2); we have not run a physical testbed. Deadlines on multipipe are scaled (Fig. 10), so absolute SLA levels are workload-

specific even when method order is stable. Offline weights come from logged trajectories rather than a fully on-policy loop [28]. The CRT-ablate experiment still omits discovery cost. Edgeward was dropped on multipipe for runtime. DynaCol itself is cited as an unpublished companion; Appendix A reprints the reused algorithms, and the public repository carries the shared runtime. Hard router failures at 5–20% are reported, but reconcile latency is mostly a control-plane timer shared across colony methods. We did not build an open-set DQN/PPO, a centralized Oracle at large N , or an energy-primary objective; energy is exported by the simulator yet is not the headline metric here. Control messages may carry service and host identifiers; securing them (authentication, encryption) is outside this study. Encoder decisions remain only partly readable through the DCBO term.

Offline cost. GraphSAGE may load a HID=16 JSON weight file and fine-tune online. That offline pass is separate from the per-decision CRT/GRT overhead in the tables; its wall-clock cost scales with the log, not with N at inference time.

8 Conclusion

Dyna-Bound ranks fog hosts inside cold-start colony tables: L2 mixes a small online scorer with DCBO, actions stay in CRT/GRT top- K , and large fabrics return to exact DCBO. On iFogSim2 multipipe workloads at $N=100$ the hybrid beats DCBO at matched overhead, with paired Holm-significant gaps at $n=20$ and medium effect sizes; the absolute SLA change is modest. GraphSAGE and the vector scorer are statistically tied, so the result is about bounded hybrid ranking. Tabular ϵ -greedy learning can lead raw SLA inside the same view and is kept as a reference; the hybrid is submitted for its DCBO anchor, large- N hand-back, and overhead contract. CRT ablations show how much QoS the bound withholds; offline import helps the GraphSAGE path; $\beta=0$ recovers DCBO. Flat baselines and node-failure trials round out the picture without changing that within-plane claim.

Natural next steps are to bill discovery traffic when the action set grows, tighten offline training toward on-policy data, and move selected scenarios onto a testbed.

Declarations

Funding. The authors declare that no funds, grants, or other support were received during the preparation of this manuscript.

Competing interests. The authors have no relevant financial or non-financial interests to disclose.

Author contributions. R.N. designed the method, implemented the experiments, and drafted the manuscript. A.G. supervised the research and revised the manuscript. Both authors read and approved the final manuscript.

Editorial tools. The authors used AI-assisted editing tools for language polishing, bibliography formatting, and LaTeX layout checks. All scientific claims, experimental designs, numerical results, and final wording were reviewed and approved by the authors.

Data availability. Simulation configs, trial CSVs, summary tables, and figure scripts are in the public JoS artifact repository <https://github.com/RouhollahNabati/Dyna-Bound> (configs/, evaluation/results/, evaluation/). The parent DynaCol / iFogSim2 runtime remains separately available at <https://github.com/RouhollahNabati/DynaCol>.

Code availability. Dyna-Bound source is available at <https://github.com/RouhollahNabati/Dyna-Bound> (Java under ifogsim2/src/org/fog/dynacolgnn/; Python tools and grids at the repository root). The unchanged DynaCol control-plane code stays in <https://github.com/RouhollahNabati/DynaCol>.

A Reused DynaCol algorithms

For self-containment relative to the prior DynaCol manuscript [34], we reprint the cold-start, handover, and DCBO L1–L3 skeletons that this paper leaves unchanged. These algorithms are *not* claimed as contributions here.

Algorithm 2 Cold-start join (Create_And_Add_FogNode)

Require: join event for node FID ; timers W_t , T_{adv} ; radius $MaxColonySize$

- 1: state $\leftarrow$ Unassigned
- 2: broadcast QueryColony(FID); start timer T
- 3: **if** no response and $T \geq W_t$ **then**
- 4: self-elect FCM; init CRT/GRT;
 AdvertisePeriodically(T_{adv})
- 5: **else**
- 6: $resp \leftarrow$ best JoinScore response
- 7: **if** $L(FID, resp.FCM) < MaxColonySize$ **then**
- 8: JoinOrNegotiate($resp$) ▷ Alg. 3
- 9: **end if**
- 10: **end if**

Algorithm 3 JoinOrNegotiate (potency-based FCM handover)

Require: response $resp$ with colony id, FCM, CRT

- 1: **if** self is FCM-candidate and Potency(self) > Potency($resp.FCM$) + ϵ and cooldown elapsed **then**
- 2: negotiate ManagerTakeoverRequest; on accept promote self and merge CRT
- 3: **else**
- 4: join $resp.ColonyID$ under $resp.FCM$; FCM updates CRT/GRT
- 5: **end if**

Algorithm 4 DCBO_PlaceService (L1–L3)

Require: service s , current host h_{cur} , tables CRT, GRT

- 1: **if** h_{cur} feasible and SLA-safe **then**
- 2: **return** h_{cur} ▷ L1 sticky
- 3: **end if**
- 4: Cand $\leftarrow$ top- K_{crt} feasible CRT hosts by $J(s, \cdot)$
- 5: **if** Cand $\neq \emptyset$ **then**
- 6: **return** arg min Cand ▷ L2 local
- 7: **end if**
- 8: **for** all col in top- K_{grt} (GRT) **do**
- 9: **if** DelegatePlace(s , col.FCM) succeeds **then**
- 10: **return** delegated host ▷ L2 neighbor
- 11: **end if**
- 12: **end for**
- 13: **return** cloud by latency/cost ▷ L3

B Hyperparameters

Table 13: Locked-grid hyperparameters.

Parameter	Default / note
CRT top- K_{crt} / GRT top- K_{grt}	5 / 3
ϵ (hybrid)	0.02 on multipipe (0 at large N)
ϵ (tabular DRL)	0.15
Vector / GraphSAGE LR	0.05 / 0.04
GraphSAGE hidden	16, 2 layers, tanh
Host / context dims	8 / 8
Blend (multipipe)	0.60
Multipipe SLA scale	10.0
Large- N gate	$n_f \geq 450 \rightarrow$ exact DCBO
Offline objective	reward-advantage BC + structure ranki
Trials	5 (tuning lock/baselines); 20 (journal N)

References

- [1] Dezheen H. Abdulazeez and Shavan K. Askar. Offloading mechanisms based on reinforcement learning and deep learning algorithms in the fog computing environment. *IEEE Access*, 11:12555–12586, 2023.
- [2] F. Ait Salaht, F. Desprez, and A. Lebre. An overview of service placement problem in fog and edge computing. *ACM Computing Surveys*, 53(3):65:1–65:35, 2020.
- [3] H. K. Apat, V. Goswami, B. Sahoo, R. K. Barik, and M. J. Saikia. Fog service placement optimization: A survey of state-of-the-art strategies and techniques. *Computers*, 14(3):99, 2025.
- [4] M. Azimzadeh, A. Rezaee, S. J. Jassbi, and M. Esnaashari. Placement of IoT services in fog environment based on complex network features: A genetic-based approach. *Cluster Computing*, 25(5):3423–3445, 2022.
- [5] A. R. Benamer, N. B. Hadj-Alouane, K. Boussetta, and A. B. Hadj-Alouane. A genetic algorithm-based approach for placement in the fog of latency-sensitive multiplayer game servers. *Cluster Computing*, 27:11249–11275, 2024.
- [6] Luiz F. Bittencourt, Javier Diaz-Montes, Rajkumar Buyya, Omer F. Rana, and Manish Parashar. Mobility-aware application scheduling in fog computing. *IEEE Cloud Computing*, 4(2):26–35, 2017.
- [7] Flavio Bonomi, Rodolfo Milito, Jiang Zhu, and Sateesh Addepalli. Fog computing and its role in the internet of things. In *Proceedings of the First Edition of the MCC Workshop on Mobile Cloud Computing*, pages 13–16, 2012.
- [8] Wassim Boudieb, Abdelhamid Malki, Mimoun Malki, Ahmed Badawy, and Mahmoud Barhamgi. Microservice instances selection and load balancing in fog computing using deep reinforcement learning approach. *Future Generation Computer Systems*, 156:77–94, 2024.
- [9] A. Brogi and S. Forti. Qos-aware deployment of IoT applications through the fog. *IEEE Internet of Things Journal*, 4(5):1185–1192, 2017.
- [10] Antonio Brogi, Stefano Forti, and Ahmad Ibrahim. How to best deploy your fog applications, probably. In *2017 IEEE 1st International Conference on Fog and Edge Computing (ICFEC)*, pages 105–114, 2017.
- [11] Omid Bushehrian and Amir Moazeni. Deep reinforcement learning-based optimal deployment of IoT machine learning jobs in fog computing architecture. *Computing*, 107(1), 2025.
- [12] Lixing Chen, Yang Bai, Pan Zhou, Youqi Li, Zhe Qu, and Jie Xu. On adaptive edge microservice placement: A reinforcement learning approach endowed with graph comprehension. *IEEE Transactions on Mobile Computing*, 23(12):11144–11158, 2024.
- [13] M. Chiang and T. Zhang. Fog and iot: An overview of research opportunities. *IEEE Internet of Things Journal*, 3(6):854–864, 2016.
- [14] Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, Hillsdale, NJ, 2 edition, 1988.
- [15] B. Costa, J. Bachiega Jr., L. R. de Carvalho, and A. P. F. Araujo. Orchestration in fog computing: A comprehensive survey. *ACM Computing Surveys*, 55(2), 2022.
- [16] Ruilong Deng, Rongxing Lu, Chengzhe Lai, Tom H. Luan, and Hao Liang. Optimal workload allocation in fog-cloud computing toward balanced delay and power consumption. *IEEE Internet of Things Journal*, 3(6):1171–1181, 2016.
- [17] J. Dogani, A. Yazdanpanah, A. Zare, and F. Khunjush. A two-tier multi-objective service placement in container-based fog-cloud computing platforms. *Cluster Computing*, 27(4):4491–4514, 2024.
- [18] Kaouther Gasmi, Selma Dilek, Suleyman Tosun, and Suat Ozdemir. A survey on computation offloading and service placement in fog computing-based IoT. *The Journal of Supercomputing*, 78(2):1983–2014, 2022.
- [19] C. Guerrero, I. Lera, and C. Juiz. Evaluation and efficiency comparison of evolutionary algorithms for service placement optimization in fog architectures. *Future Generation Computer Systems*, 97:131–144, 2019.
- [20] Harshit Gupta, Amir Vahid Dastjerdi, Soumya K. Ghosh, and Rajkumar Buyya. iFogSim: A toolkit for modeling and simulation of resource management techniques in the internet of things, edge and fog computing environments. *Software: Practice and Experience*, 47(9):1275–1296, 2017.
- [21] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [22] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- [23] C.-H. Hong and B. Varghese. Resource management in fog/edge computing: A survey on architectures, infrastructure, and algorithms. *ACM Computing Surveys*, 52(5), 2019.

- [24] H. S. Hossam, H. Abdel-Galil, and M. Belal. An energy-aware module placement strategy in fog-based healthcare monitoring systems. *Cluster Computing*, 27:7351–7372, 2024.
- [25] Parviz Keshavarz Haddadha, Mohammad Hossein Rezvani, Mahdi MollaMotallebi, and Achyut Shankar. Machine learning methods for service placement: A systematic review. *Artificial Intelligence Review*, 57(3):61, 2024.
- [26] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.
- [27] I. Lera and C. Guerrero. Multi-objective application placement in fog computing using graph neural network-based reinforcement learning. *The Journal of Supercomputing*, 80:27073–27094, 2024.
- [28] Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. arXiv:2005.01643, 2020.
- [29] R. Mahmud, K. Ramamohanarao, and R. Buyya. Latency-aware application module management for fog computing environments. *ACM Transactions on Internet Technology*, 19(1):1–21, 2018.
- [30] Redowan Mahmud, Samodha Pallewatta, Mohammad Goudarzi, and Rajkumar Buyya. iFogSim2: An extended iFogSim simulator for mobility, clustering, and microservice management in edge and fog computing environments. *Journal of Systems and Software*, 190:111351, 2022.
- [31] Redowan Mahmud, Kotagiri Ramamohanarao, and Rajkumar Buyya. Application management in fog computing environments: A taxonomy, review and future directions. *ACM Computing Surveys*, 53(4):43:1–43:43, 2020.
- [32] V. Mnih et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.
- [33] Carla Mouradian, Diala Naboulsi, Sami Yangui, Roch H. Glitho, Monique J. Morrow, and Paul A. Polakos. A comprehensive survey on fog computing: State-of-the-art and research challenges. *IEEE Communications Surveys & Tutorials*, 20(1):416–464, 2018.
- [34] R. Nabati and A. Ghaderzadeh. DynaCol: Self-organizing fog colonies with cold-start formation and bounded resource tables. Unpublished manuscript, 2026. Control-plane companion; not submitted to Cluster Computing.
- [35] Ranesh Kumar Naha, Saurabh Garg, Dimitrios Georgakopoulos, Prem Prakash Jayaraman, Longxiang Gao, Yong Xiang, and Rajiv Ranjan. Fog computing: Survey of trends, architectures, requirements, and research directions. *IEEE Access*, 6:47980–48009, 2018.
- [36] Tao Ouyang, Zhi Zhou, and Xu Chen. Follow me at the edge: Mobility-aware dynamic service placement for mobile edge computing. *IEEE Journal on Selected Areas in Communications*, 36(10):2333–2345, 2018.
- [37] S. Pallewatta, V. Kostakos, and R. Buyya. Placement of microservices-based iot applications in fog computing: A taxonomy and future directions. *ACM Computing Surveys*, 55(14s):321:1–321:43, 2023.
- [38] Stéphane Ross, Geoffrey Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 627–635, 2011.
- [39] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv:1707.06347, 2017.
- [40] O. Skarlat, M. Nardelli, S. Schulte, M. Borkowski, and P. Leitner. Optimized IoT service placement in the fog. *Service Oriented Computing and Applications*, 11(4):427–443, 2017.
- [41] O. Skarlat, M. Nardelli, S. Schulte, and S. Dustdar. Towards qos-aware fog service placement. In *Proc. IEEE 1st International Conference on Fog and Edge Computing (ICFEC)*, pages 89–96, 2017.
- [42] Sven Smolka and Zoltán Ádám Mann. Evaluation of fog application placement algorithms: A survey. *Computing*, 104(6):1397–1423, 2022.
- [43] C. Sonmez, A. Ozgovde, and C. Ersoy. Edgecloudsim: An environment for performance evaluation of edge computing systems. *Transactions on Emerging Telecommunications Technologies*, 29(11):e3493, 2018.
- [44] T. Supraja, P. Chawla, and S. S. Gill. AI-Driven Service Placement in Fog and Edge Computing Environments: A Systematic Literature Review, Taxonomy and Future Directions. *Cluster Computing*, 28(16):1070, 2025.
- [45] I. Taleb, J.-L. Guillaume, and B. Duthil. A survey on services placement algorithms in integrated cloud-fog/edge computing. *ACM Computing Surveys*, 57(11), 2025.
- [46] Mohit Taneja and Alan Davy. Resource aware placement of IoT application modules in fog-cloud computing paradigm. In *2017 IFIP/IEEE Symposium*

- on *Integrated Network and Service Management (IM)*, pages 1222–1228, 2017.
- [47] S. Tuli, N. Basumatary, S. S. Gill, M. Kahani, R. C. Arya, G. S. Wander, and R. Buyya. HealthFog: An ensemble deep learning based smart healthcare system for automatic diagnosis of heart diseases in integrated IoT and fog computing environments. *Future Generation Computer Systems*, 104:187–200, 2020.
 - [48] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.
 - [49] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)*, 2019.
 - [50] A. Yousefpour, C. Fung, T. Nguyen, K. Kadiyala, F. Jalali, A. Niakanlahiji, J. Kong, and J. P. Jue. All one needs to know about fog computing and related edge computing paradigms: A complete survey. *Journal of Systems Architecture*, 98:289–330, 2019.
 - [51] Ashkan Yousefpour, Ashish Patil, Genya Ishigaki, Inwoong Kim, Xi Wang, Hakki C. Cankaya, Qiong Zhang, Weisheng Xie, and Jason P. Jue. FOGPLAN: A lightweight QoS-aware dynamic fog service provisioning framework. *IEEE Internet of Things Journal*, 6(3):5080–5096, 2019.